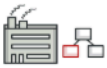
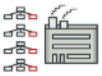
Adapter Pattern 3
Facade Pattern 5
Proxy Pattern 6
Decorator Pattern 8
Composite Pattern 10
Bridge Pattern 12

Creational patterns

These patterns provide various object creation mechanisms, which increase flexibility and reuse of existing code.

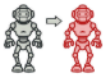


Factory Method



Abstract Factory

Builder

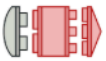


Prototype

Singleton

Structural patterns

These patterns explain how to assemble objects and classes into larger structures while keeping these structures flexible and efficient.



Adapter

Bridge

Composite

Decorator

Facade

Flyweight

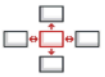
Behavioral patterns

These patterns are concerned with algorithms and the assignment of responsibilities between objects.

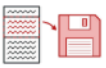
Chain of Responsibility

Command

Iterator

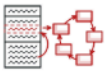


Mediator



Memento

Observer



State

Strategy

Template Method

Visitor

🧩 Résumé des Structural Design Patterns

Pattern	But principal	Exemple simple	Cas d'usage typique
Adapter	Adapter une interface incompatible à ce que le client attend	`LegacyPdfAdapter` qui convertit une méthode existante en `export()`	Intégrer une vieille API à ton code moderne
Facade	Fournir une **interface simple** à un **système complexe**	`TravelBookingFacade` qui appelle `HotelService`, `FlightService`, etc.	Cacher plusieurs appels internes dans un seul `registerClient()`
Proxy	**Contrôler l'accès** à un objet réel ou **intercepter** les appels	`SecureDocumentProxy` ou `LoggingProxy`	Sécurité, cache, lazy loading, audit
Decorator	**Ajouter dynamiquement des fonctionnalités** sans modifier l'objet	`EncryptedNotifier`, `SoundNotifier`	Logs, compression, validation, enrichissement
Composite	Représenter une **structure arborescente récursive**	`Folder` contenant des `File` ou d'autres `Folder`	Arborescence UI, organisation entreprise, fichier/dossier
Bridge	**Séparer l'abstraction de l'implémentation** pour les faire évoluer indépendamment	`Notification` + `MessageSender`	Combiner formes avec styles de rendu, notification avec canaux

Adapter Pattern

🔹 Problème :

Tu veux **réutiliser une classe existante** (ou une lib) dans un système…
Mais **son interface n'est pas compatible** avec ce que tu attends.

🔌 Solution :

Créer un **Adapter** qui **fait le pont** entre l'interface attendue (client) et celle de la classe existante (service).

📘 Exemple simple :

- Tu as un système qui attend une méthode `payWithCard()`

- Mais ta classe de paiement existante a `processVisa()`
  ➡️ Tu crées un `PaymentAdapter` qui **adapte `processVisa()` en `payWithCard()`**

Élément	Détail
But	Rendre compatible une interface existante avec une interface attendue
Structure	`Target` (interface), `Adapter`, `Adaptee` (classe existante)
Type courant en Java	Object Adapter (composition)

Créer une interface → c'est l'interface cible attendue par ton système.
Avoir une classe/service existant → c'est l'adaptee, que tu ne veux/pas modifier.
Créer un adapter :

- qui implémente l'interface
- et délègue à l'instance du service existant via injection / composition

```
//Target
public interface PaymentProcessor {  2 usages
    void pay(String customer, double amount);  1 usage
}
//Adaptee
public static class LegacyPaymentService {  4 usages
    public void makeTransaction(String client, double value) {  1 usage
        System.out.println("Legacy processing payment of " + value + "€ for " + client);
    }
}
```

```java
//Adapter
public static class LegacyPaymentAdapter implements PaymentProcessor {  1 usage
    private LegacyPaymentService legacyService;  2 usages

    public LegacyPaymentAdapter(LegacyPaymentService legacyService) {  1 usage
        this.legacyService = legacyService;
    }

    @Override  1 usage
    public void pay(String customer, double amount) {
        legacyService.makeTransaction(customer, amount);
    }
}
```

Facade Pattern

🔹 Problème :

Tu as un sous-système complexe (plusieurs classes qui interagissent entre elles), et tu veux offrir une interface simple au client pour tout gérer.

🔧 Solution :

Créer une **classe façade** qui **regroupe** les appels vers les différentes classes internes et expose une **API simple et claire**.

Sans Façade :

```java
BookingSystem booker = new BookingSystem();
PaymentProcessor payment = new PaymentProcessor();
NotificationService notify = new NotificationService();

// 3 appels complexes
booker.reserve();
payment.pay();
notify.sendConfirmation();
```

Avec Façade :

```java
TravelAgencyFacade facade = new TravelAgencyFacade();
facade.bookTrip(); // 1 seul appel
```

Cela respecte le principe de responsabilité unique (SRP), et cache la complexité métier derrière une interface claire.

Proxy Pattern

🔹 But :

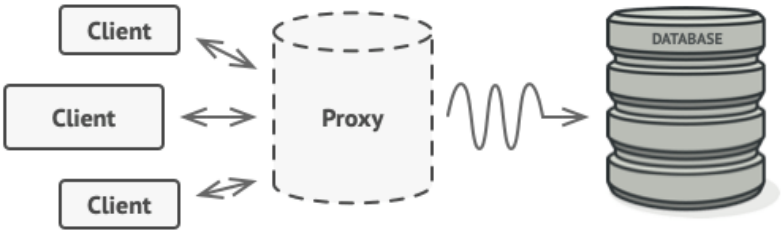
Le Proxy agit comme un intermédiaire entre le client et un objet réel.
Il intercepte les appels et peut :

- ajouter des règles d'accès
- faire du cache
- enregistrer des logs
- différer l'exécution (lazy loading)

⚠️ Le client ne sait pas s'il parle au vrai objet ou à un proxy.

✅ Types de proxy :

Type	Rôle
Virtual Proxy	Chargement paresseux (`lazy load`)
Protection Proxy	Contrôle d'accès
Logging Proxy	Ajouter des logs
Remote Proxy	Accès à distance (ex: RMI, WebService)



The proxy disguises itself as a database object. It can handle lazy initialization and result caching without the client or the real database object even knowing.

Le Proxy **délègue l'appel** à un objet réel, mais peut **intercepter** pour ajouter des comportements : logs, sécurité, cache…

Élément	Détail
But	Intercepter les appels vers un objet réel
Types	Virtual, Protection, Logging, Remote
Structure	Interface, Objet réel, Proxy implémentant la même interface

🚀 En Spring, on utilise le Proxy Pattern principalement dans :

1. AOP (Aspect-Oriented Programming)

➡️ Spring intercepte les appels de méthode pour ajouter :`@Transactional`, `@Async`, `@Cacheable`, `@Scheduled`

2. Injection de dépendances (DI)

Tu as	Spring fait
`@Service` + `@Transactional`	Crée un **proxy** qui ajoute la transaction
`@Autowired` d'une interface	Crée un **JDK proxy**
`@Autowired` d'une classe	Crée un **CGLIB proxy** (héritage)

```java
interface ProductService {
    String getProductDetails(String productId) throws InterruptedException;
}

// Le vrai service (simule un traitement long)
static class RealProductService implements ProductService {
    @Override
    public String getProductDetails(String productId) throws InterruptedException {
        System.out.println("Fetching product details from RealProductService...");
        Thread.sleep(1000); // Simule un appel coûteux
        return "Product details for " + productId;
    }
}
```

```java
// Le proxy qui met en cache
static class CachingProductProxy implements ProductService {
    private final ProductService realService = new RealProductService();
    private final Map<String, String> cache = new HashMap<>();

    @Override
    public String getProductDetails(String productId) throws InterruptedException {
        if (cache.containsKey(productId)) {
            System.out.println("Returning cached result for " + productId);
            return cache.get(productId);
        } else {
            String productDetail = realService.getProductDetails(productId);
            cache.put(productId, productDetail);
            return productDetail;
        }
    }
}
```

Decorator Pattern

🔹 But :

Permet d'**ajouter dynamiquement des fonctionnalités** à un objet **sans modifier son code**.

⚠️ Contrairement au Proxy (qui contrôle ou protège), **le Decorator enrichit** l'objet d'origine avec de nouveaux comportements.

Élément	Détail
But	Ajouter dynamiquement des responsabilités
Structure	Interface → composant de base + décorateurs
Avantage	Pas besoin de modifier les classes existantes
Utilisation	Chiffrement, logs, compression, validation, etc.

```java
interface Notifier {
    String send(String message);
}

// 1. Composant de base
static class BasicNotifier implements Notifier {
    @Override
    public String send(String message) {
        return "Sending message: " + message;
    }
}
```

`wrapped` **Le composant "intérieur" à qui on délègue**

```java
// 2. Decorator : Encryption
static class EncryptedNotifier implements Notifier {
    private final Notifier wrapped;

    public EncryptedNotifier(Notifier wrapped) {
        this.wrapped = wrapped;
    }

    @Override
    public String send(String message) {
        String result = wrapped.send(message);
        return "Encrypted(" + result + ")";
    }
}
```

```java
// 3. Decorator : Sound
static class SoundNotifier implements Notifier {
    private final Notifier wrapped;

    public SoundNotifier(Notifier wrapped) {
        this.wrapped = wrapped;
    }

    @Override
    public String send(String message) {
        String result = wrapped.send(message);
        return "Sound -> " + result;
    }
}
```

```java
Notifier notifier = new SoundNotifier(new EncryptedNotifier(new
BasicNotifier()));
System.out.println(notifier.send("Hello"));
```

Sound -> Encrypted(Sending message: Hello)

Composite Pattern

Un très beau pattern structurel, puissant quand tu veux représenter des **objets en arborescence**, comme :

- un **menu avec sous-menus**
- une **organisation avec employés et managers**
- des **dossiers et fichiers**
- ou encore des **formulaires imbriqués**

🔹 But :

Permet de **traiter de manière uniforme** des objets simples et des **groupes d'objets**, grâce à une structure **récursive**.

le Composite: Un conteneur d'autres objets (feuilles ou composites)
Le `Composite` est **un conteneur récursif** : il contient soit des **`Leaf`**, soit d'autres **`Composite`**.
Par exemple : un `Folder` peut contenir des fichiers **et** d'autres dossiers.

Élément	Exemple
`Component`	Interface `FileSystemItem`
`Leaf`	Fichier (`File`)
`Composite`	Dossier (`Folder`) contenant d'autres items

EXP

Tu veux modéliser une structure d'entreprise.
Un **employé** peut être :

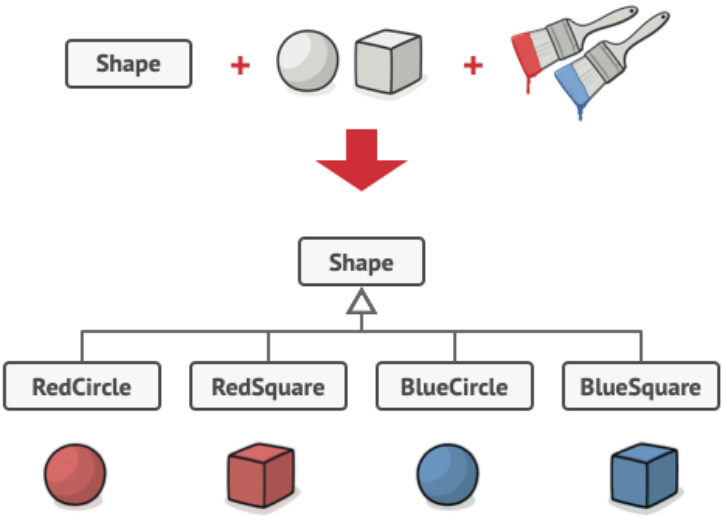
- un **employé individuel** (`SimpleEmployee`)
- un **manager** (`Manager`) qui **gère une équipe** (d'autres employés ou managers)

```java
public class Composite {

    interface Employee {
        void display(String indent);
    }

    // 1. Feuille
    static class SimpleEmployee implements Employee {
        private final String name;

        public SimpleEmployee(String name) {
            this.name = name;
        }

        @Override
        public void display(String indent) {
            System.out.println(indent + "Employee: " + name);
        }
    }
```

```java
    // 2. Composite
    static class Manager implements Employee {
        private final String name;
        private final List<Employee> employees = new ArrayList<>();

        public Manager(String name) {
            this.name = name;
        }

        public void add(Employee employee) {
            employees.add(employee);
        }

        @Override
        public void display(String indent) {
            System.out.println(indent + "Manager: " + name);
            for (Employee e : employees) {
                e.display(indent + "  ");
            }
        }
    }
```

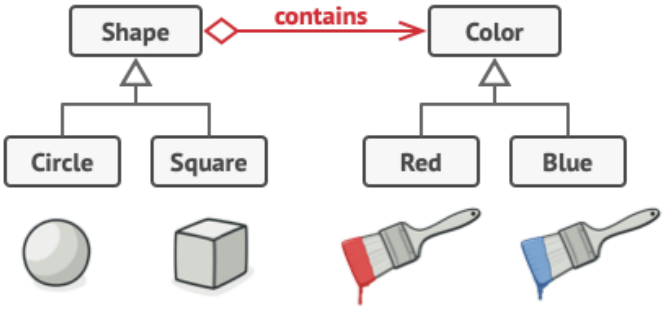
Bridge Pattern

Say you have a geometric `Shape` class with a pair of subclasses: `Circle` and `Square`. You want to extend this class hierarchy to incorporate colors, so you plan to create `Red` and `Blue` shape subclasses. However, since you already have two subclasses, you'll need to create four class combinations such as `BlueCircle` and `RedSquare`.



Number of class combinations grows in geometric progression.

This problem occurs because we're trying to extend the shape classes in two independent dimensions: **by form and by color.** That's a very common issue with class inheritance.



You can prevent the explosion of a class hierarchy by transforming it into several related hierarchies.

Le Bridge Pattern permet de séparer une abstraction (ex: une forme) de son implémentation (ex: la façon dont elle est dessinée) pour qu'elles puissent évoluer indépendamment.
C'est parfait pour éviter l'explosion de sous-classes.
L'abstraction contient une référence vers l'implémentation.

Tu veux envoyer des **notifications** de différents types :

- `Alert`
- `Reminder`
- `Warning`

Mais tu veux pouvoir **les envoyer par différents canaux** :

- `EmailSender`
- `SmsSender`

➡️ Tu ne veux pas créer `EmailAlert`, `SmsAlert`, `EmailReminder`, `SmsReminder`, etc. ❌
➡️ Tu veux **séparer le type de notification** de la **manière de l'envoyer** ✅
➡️ → Cas parfait pour le **Bridge Pattern**

```java
public class Bridge {
    // 1. Interface implémentation (Bridge)
    interface MessageSender{  6 usages  2 implementations
        void send(String content);  2 usages  2 implementations
    }
    static class EmailSender implements MessageSender{  1 usage
        @Override  2 usages
        public void send(String content){
            System.out.println("Sending by email: "+content);
        }
    }
    static class SmsSender implements MessageSender{  1 usage

        @Override  2 usages
        public void send(String content) {
            System.out.println("Sending by sms: "+content);
        }
    }
```

```java
// 2. Abstraction
abstract static class Notification{  4 usages  2 inheritors
    protected final MessageSender sender;  3 usages
    Notification(MessageSender sender) {  2 usages
        this.sender = sender;
    }
    // méthode abstraite à implémenter
    public abstract void notify(String content);  2 usages  2 implementations
}
```

```java
// 3. Abstractions concrètes
static class AlertNotification extends Notification {  1 usage
    AlertNotification(MessageSender sender) {  1 usage
        super(sender);
    }
    @Override  2 usages
    public void notify(String content) {
        sender.send( content: "Alert: " + content);
    }
}

static class ReminderNotification extends Notification {  1 usage
    ReminderNotification(MessageSender sender) {  1 usage
        super(sender);
    }

    @Override  2 usages
    public void notify(String content) {
        sender.send( content: "Reminder: " + content);
    }
}
```

```java
public static void main(String[] args) {
    Notification notif1 = new AlertNotification(new SmsSender());
    notif1.notify( content: "Server down!");

    Notification notif2 = new ReminderNotification(new EmailSender());
    notif2.notify( content: "Meeting at 15:00");

}
```